There are a number of strategies for accepting and rejecting CORS requests, depending on how open or closed you'd like your server to be. We cover these strategies in chapter 6.

3.6 Summary

This chapter provided an overview of how CORS works from the server's perspective. I started by defining the players in a CORS request:

- The client, which initiates the cross-origin request
- The browser, which manages the communication between the client and the server
- The server, which serves data that the client wants

Next, I covered the HTTP headers needed for a basic CORS request:

- The browser sends the Origin header to indicate where a request is coming from.
- An origin is defined as the scheme, host, and port portion of a URL.
- The server responds with the Access-Control-Allow-Origin header if the request is valid.

Finally, you learned that the Access-Control-Allow-Origin header supports two values:

- Setting the Access-Control-Allow-Origin header to * allows cross-origin requests from any client.
- Setting the Access-Control-Allow-Origin header to a specific origin value only allows cross-origin requests from that specific client.

The techniques described in this chapter should give you a good understanding of how CORS works, and how to add simple CORS support to a server. But not all requests can be handled with only the Access-Control-Allow-Origin header. More complex HTTP requests, like `PUT` or `DELETE`, or requests with custom HTTP headers, will still fail. Any cross-origin request beyond the simplest request needs additional processing to succeed. We'll cover these new processing techniques in the next chapter.